CS288 Assignment: Retrieval-Augmented Generation System

Ziyu Ge
ziyuge@berkeley.edu

Q1. QA Data Creation.

We curated a QA dataset from UC Berkeley EECS-related webpages (official site, legacy `www2`, course catalogs, faculty profiles, advising/policy pages, research pages, and announcements). We designed multiple types of questions: (i) **Yes/No questions** (e.g., eligibility rules, policy constraints), (ii) **Count-based questions** (e.g., number of courses, faculty, or programs), (iii) **Date/time questions** (e.g., deadlines, event schedules), and (iv) **Entity lookup and multi-hop questions** (e.g., identifying advisors, combining information in page such as faculty + awards). In total, it contains **112 questions**. Inter-annotator agreement (IAA), measured on 30 randomly sampled questions, reached **93% exact match**, with most disagreements arising from minor formatting differences (e.g., "Spring 2026" vs. "Sp26").

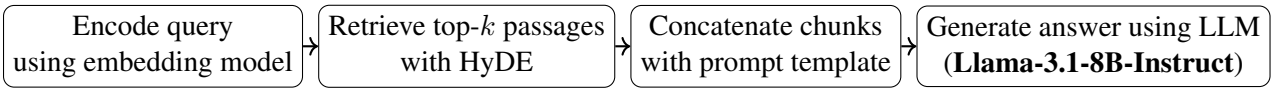
Examples: Q1: How many COMPSCI 198 group courses are offered in Spring 2026? A1: 5; Q2: Who is the CS student advisor? A2: Gina Garcia; Q3: What is the course number for the class covering cryptography and security? A3: CS 161

Q2. Retrieval Corpus.

We built an asynchronous EECS-domain web crawler. To bypass the legacy `www2` subdomain's aggressive WAF (HTTP 429s), we engineered a **Global Async WAF Lock** that instantly pauses all concurrent workers upon hitting a rate limit to prevent IP bans. The scraped HTML was stripped of boilerplate to improve signal quality and chunked into overlapping $\sim$200-token passages.

We evaluated the corpus via manual inspection, coverage, and downstream RAG performance. Compared to the provided reference corpus, ours achieved similar recall but slightly higher Exact Match (EM) on structured count-based queries. We hypothesize the reference corpus remains more robust on long-tail queries due to broader web coverage and stricter deduplication.

Q3. RAG System.



HyDE (Hypothetical Document Embeddings): For dense retrieval, the query is first passed to the LLM to generate a short hypothetical answer. This bridges the semantic gap between question phrasing and answer phrasing in embedding space. BM25 runs independently on the original query (top-40).

Prompt Engineering: It enforces strict brevity: the model must output the shortest possible span that answers the question, with explicit rules for each answer type. The prompt also includes a strong anti-refusal clause — the model is explicitly told it must attempt to answer, and may only output "I don't know" if the context contains absolutely no relevant information.

Q4. Ablations.

Ablation 1 & 2: Corpus Refinement and Model Selection

Early corpus iterations suffered from WAF rate-limiting and boilerplate noise. By implementing our global WAF lock and aggressive HTML cleaning, our final `corpus_raw_5` drastically improved quality and was locked in for all experiments. Using this corpus, we evaluated open-weight models ($\leq$8B) across $k \in \{5, 10, 15\}$, optimizing for extractive accuracy and latency.

Table 1: Generator Performance and System Recall across Top-k

	$k = 5$ (Recall: 0.8125)		$k = 10$ (Recall: 0.8393)		$k = 15$ (Recall: **0.8571**)	
Model	**EM**	**F1**	**EM**	**F1**	**EM**	**F1**
`llama-3.1-8b-instruct`	**0.4821**	**0.6459**	**0.5446**	**0.7048**	**0.5268**	**0.6913**
`allenai/olmo-3-7b`	**0.4821**	0.6361	0.4732	0.6126	0.1696	0.2315
`qwen/qwen3-8b`	0.4286	0.5460	0.3482	0.5147	0.4375	0.5953
`mistralai/mistral-7b`	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

`llama-3.1-8b-instruct` was the clear winner, outperforming its predecessor (`llama-3`) by $\sim$14.5% F1. While `qwen3-8b` is cited as state-of-the-art, its severe latency ($\sim$1631s) violates the autograder constraints. `mistral-7b` failed entirely (0.0 EM/F1) due to silent deprecation on the OpenRouter API.

Fine-Grained Tuning: Llama 3 vs. 3.1

We ran a fine-grained evaluation to map the inflection point where context noise outweighs retrieval recall.

Table 2: Fine-Grained F1 Score Comparison

Model	$k = 6$	$k = 8$	$k = 10$	$k = 12$	$k = 14$
`llama-3.1-8b-instruct`	0.6630	0.6987	0.7048	**0.7060**	0.6969
`llama-3-8b-instruct`	0.5915	0.5597	-	0.5871	0.5553

`llama-3.1` massively outperforms `llama-3` across all windows (+0.139 F1 at $k = 8$). For `llama-3.1`, pushing past $k = 12$ causes F1 to drop (0.6969 at $k = 14$), confirming the "lost in the middle" phenomenon as context noise accumulates. Eventually, we selected `llama-3.1-8b-instruct` at $k = 10$ as it captures 99.8% of the peak F1 observed at $k = 12$ while processing 16% fewer context tokens, maximizing our robustness against the autograder's strict latency limits.

Q5. Error Analysis.

Table 3: Classification of 36 zero-F1 answers in Hidden Dev Set.

Class	**Count**	**Cause**	**Representative Examples**
Retrieval Miss	10	Correct chunk absent from retrieved context; model hallucinates or refuses.	"Who earned a PhD in 1935?" $\rightarrow$ *I don't know*. GT: *Charles F. Dalziel*
Over-Refusal	10	Retrieval succeeded but model outputs *I don't know*; often occurs on counting.	"Is signal processing in CS 70?" $\rightarrow$ *I don't know*, 12 matching chunks, GT: *No*
Hallucination on Context	12	Retrieved correctly but model selects wrong value: competing numbers, wrong list member, or fails to compute derived values.	"How long ago did Malik do his PhD?" $\rightarrow$ *1985*, model gives year, not elapsed time, GT: *41*
Metric False Negative	4	Model answer is semantically correct but token-level F1 is 0 due to form differences.	Phone: *510-642-3214*, GT: *1(510) 642-3214*

Additionally, 10 partial-F1 cases suffer softer metric issues (e.g., *"1 year"* vs. *"one year"*, F1 = 0.50). The current F1 assumes both GT and prediction are extracted from the same source text. The most impactful fix would be number/unit normalization.

Q6. Takeaways & Future Work.

Because the evaluation relies on strict Exact Match and token-level F1, even minor conversational hallucinations drastically penalize the score. Furthermore, we proved empirically that naively increasing k degrades F1 due to context-noise accumulation. With more time, we would focus on three improvements: incorporating hybrid retrieval (dense + sparse) for exact entity matches; optimizing latency (quantization/vLLM) to enable stronger models; and adopting semantic-aware evaluation (LLM-as-a-Judge) to reduce penalties from rigid scoring.

GenAI Statement

We used generative AI tools to improve writing clarity, formatting, and assist with coding. All core technical design, experiments, and analysis were conducted by the authors.

References

- **Models:** AI@Meta (2024). *The Llama 3 Herd of Models* (Llama 3.1 8B Instruct).
- **Libraries:** Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* (FAISS).
- **Data & Corpus:** UC Berkeley EECS Web Domains (`eecs.berkeley.edu`, `www2.eecs.berkeley.edu`).
- **Courseware:** UC Berkeley CS 288 (Spring 2026) Assignment 3 guidelines (inspired by CMU CS11-711).

Appendix: From Early Milestone to Final Submission

Milestone	Dev F1	Recall	Test F1	Key Change
Baseline	47.18%	67%	—	Initial system, noisy corpus
Corpus Cleanup	59.21%	79%	—	Clean corpus, index rebuild
k Tuning + Prompt	62.26%	80%	55.76%	k=12, extractive prompt rules
Deduplication + Pool	59.95%	81%	55.03%	Dedup, wider candidate pool
BGE + HyDE (final)	**62.66%**	**88%**	**61.00%**	BGE-base embeddings, HyDE, k=20